

VelocityAI: Hardware-Software Co-Design for Efficient Edge LLM Inference and Shared-Memory Multi-Agent Systems

Mrutyunjay Joshi

Independent Systems Developer & ML Researcher

Project Repository: <https://github.com/mrutyunjay11/velocityai>

Abstract

Edge deployment of Large and Small Language Models (SLMs) is fundamentally constrained by memory bandwidth, initialization latency, and operating system scheduling jitter. While existing frameworks optimize individual runtime components (e.g., quantization, GPU offloading, or general tensor execution), they do not jointly address cold-start weight ingestion, CPU synchronization overhead, non-quantized decoding, and model-weight duplication during sequential multi-agent collaboration. In this paper, we present **VelocityAI**, a lightweight inference engine and multi-agent system co-designed for Apple Silicon’s Unified Memory Architecture (UMA). VelocityAI makes five primary contributions: (1) a **zero-copy model ingestion architecture** via POSIX `mmap` that eliminates redundant user-space copying, achieving a cold-start latency of **0.14 s** (a $5.6\times$ reduction relative to Apple MLX and a $41.9\text{--}72.1\times$ reduction relative to PyTorch); (2) a **vectorized C++17 ARM NEON SIMD pipeline** with loop unrolling and prefetching; (3) a **persistent spin-barrier worker pool** reducing thread synchronization latency from $12.4\mu\text{s}$ to $0.28\mu\text{s}$, achieving **41.4–48.2 tokens/second** on multi-core CPU in non-quantized Float16 precision; (4) an **algorithmic cycle-recovery safeguard** that intercepts greedy attractor loops and recovers reasoning contexts; and (5) a **shared-memory multi-agent runtime** that executes sequential multi-role pipelines with **0 MB of additional model-weight duplication**. We provide empirical benchmarks, ablations, and statistical evaluations on Apple Silicon M2, demonstrating superior responsiveness and memory scalability.

Keywords: LLM Inference, Edge Computing, Apple Silicon, Unified Memory Architecture, ARM NEON SIMD, Multi-Agent Systems, Systems for Machine Learning.

1 Introduction

1.1 Motivation

The rapid proliferation of Small Language Models (SLMs)—such as SmolLM2 [1], Qwen2.5 [2], and Llama-3.2 [3]—has made local, on-device artificial intelligence practical on consumer personal computers. Local execution guarantees strict data privacy, eliminates per-token API costs,

operates air-gapped without internet access, and delivers predictable offline latency.

1.2 Problem Statement

Executing autoregressive transformer inference on consumer hardware exposes four distinct system-level bottlenecks:

- 1. Cold-Start Weight Ingestion Penalty:** In interactive command-line utilities, short-lived subprocesses, and automated scripts, total task latency is dominated by weight deserialization. General-purpose frameworks (e.g., PyTorch) allocate temporary memory buffers and copy weights into heap objects, taking 5.8–10.1 seconds before decoding begins.
- 2. Memory-Bandwidth Bottleneck in Autoregressive Decoding:** Single-token generation ($N = 1$) is strongly memory-bandwidth-bound because a large fraction of the model parameters must be read from memory into processor caches for each generated token, while the arithmetic work per byte is comparatively low. Under Float16 precision, the arithmetic intensity is $\mathcal{I} \approx 1 \text{ FLOP / Byte}$, making memory access speed and low-overhead thread synchronization the primary determinants of throughput [7].
- 3. Thread Scheduling Context-Switch Overhead:** Conventional multi-core runtimes coordinate parallel workers using POSIX condition variables (`pthread_cond_wait`) and kernel mutexes. In high-frequency transformer execution (where each layer requires $10\text{--}30\mu\text{s}$ across 32 layers), operating system thread sleep and wakeup transitions ($8\text{--}15\mu\text{s}$ per barrier) consume 10–20% of total inference time.
- 4. Model-Memory Duplication in Multi-Agent Pipelines:** Emerging multi-agent architectures (e.g., AutoGen [6], CrewAI) partition tasks across collaborative specialized personas (e.g., Architect $\rightarrow$ Developer $\rightarrow$ Reviewer). Standard frameworks spawn isolated processes or redundant model instances for each role, linearly multiplying system RAM and triggering swap pressure on edge devices.

1.3 Research Gap and Research Questions

Existing local inference systems optimize individual components of the inference pipeline, including quantized computation, GPU acceleration, or general-purpose tensor execution. However, these approaches do not simultaneously address cold-start weight ingestion, CPU synchronization overhead, non-quantized CPU decoding, and model-memory duplication during sequential multi-agent execution. VelocityAI investigates whether these system-level bottlenecks can be jointly reduced through a hardware-aware inference architecture specifically designed around Apple Silicon’s unified memory model.

Specifically, this paper investigates five concrete **Research Questions (RQs)**:

- **RQ1 (Cold Start):** Does zero-copy virtual memory mapping significantly reduce model cold-start ingestion latency compared to heap deserialization?
- **RQ2 (CPU Kernel Efficiency):** How much do ARM NEON vectorization, loop unrolling, and prefetching improve CPU decoding throughput over scalar baselines?
- **RQ3 (Synchronization Latency):** Does persistent spin-barrier worker synchronization reduce synchronization jitter and barrier latency relative to OS primitives?
- **RQ4 (Multi-Agent Memory Scaling):** Can shared model-weight memory eliminate redundant weight allocations during sequential multi-agent workflows?
- **RQ5 (Decoding Stability):** Can multiscale cycle detection prevent degenerative generation traps without causing false-positive interruptions?

1.4 Contributions

The primary contributions of this work are:

1. **A Zero-Copy Model Ingestion Architecture** that maps SafeTensors binary files directly into process virtual memory using POSIX `mmap` and binds tensor views via deterministic header offsets.
2. **An Optimized ARM NEON CPU Inference Pipeline** combining FP16 vector multiply-accumulate intrinsics, 32-wide unrolling, software prefetching, and persistent worker synchronization with user-interactive QoS scheduling.
3. **A Multi-Scale Decoding Safeguard** that intercepts greedy repetition loops across consecutive, alternating, and high-density patterns, with in-flight reasoning context recovery.
4. **A Shared-Memory Multi-Agent Execution Architecture** in which multiple role-specific agents execute sequentially over a single shared model instance, achieving 0MB of additional model-weight duplication.

5. **An Empirical Evaluation and Ablation Study** systematically benchmarking cold-start latency, decode speed, synchronization latency, and memory scaling against PyTorch, Apple MLX, and Ollama on physical Apple Silicon hardware.

2 Related Work

2.1 Edge LLM Inference & System Run-times

Local language model execution has evolved rapidly with frameworks like `llama.cpp` [4], which pioneered edge LLM execution using quantized integer representations (Q4_K_M, Q8_0) across CPU SIMD extensions (AVX2, NEON) and GPU backends. While effective for resource-constrained systems, integer quantization causes non-trivial perplexity degradation in small-parameter models ($\leq 1\text{B}$ parameters), frequently leading to syntactical errors in code generation. PyTorch [8] and ExecuTorch provide flexible research execution but incur significant memory footprints and initialization delays.

2.2 Apple Silicon UMA & Apple MLX

Apple Silicon integrates CPU cores, a Metal GPU, and the Apple Neural Engine onto a Unified Memory Architecture (UMA), sharing high-bandwidth LPDDR5 memory (100–400+ GB/s). Apple MLX [5] exploits UMA through lazy array evaluation and shared GPU buffers. However, MLX requires Python-to-C++ graph dispatch layers, which incur modest startup overheads (0.79s) and focus primarily on Metal GPU execution rather than lightweight CPU deployment.

2.3 Memory-Mapped Weight Loading

POSIX `mmap` has been used in file-backed database engines and systems programming to bypass user-space memory allocations. In deep learning runtimes, SafeTensors introduced contiguous tensor serialization. However, standard loaders frequently read and duplicate mapped arrays into framework-specific tensor structures, discarding the cold-start advantages of direct pointer binding.

2.4 SIMD Vectorization & Worker Synchronization

Modern CPU architectures feature vector SIMD extensions (such as ARM NEON and AVX-512) capable of accelerating linear algebra routines. Prior work on multi-core parallelization relies on OS-managed threadpools. However, when operating under microsecond-scale task granularities (e.g., individual layer GEMV steps), the OS context-switch penalty becomes a primary bottleneck.

Table 1: Comprehensive Edge Inference Benchmark on Apple Silicon M2 (SmolLM2-360M-Instruct, 16 GB Unified Memory, $n = 20$ runs)

Framework / Engine	Precision Format	Hardware Device	Cold-Start Latency (s)	Decode Speed (tok/s)	Active RAM (RSS)	Speedup vs PyTorch
PyTorch 2.x (Hugging Face)	Float16 (Non-quantized)	Apple M2 CPU	7.98 ± 1.42 (5.86–10.09)	15.2 ± 0.8	~1,850 MB	1.0× (baseline)
Ollama 0.5.x (llama.cpp)*	4-bit Quantized (Q4_K_M)	Apple M2 CPU (NEON)	1.10 ± 0.08	42.1 ± 1.1	~650 MB	7.3×
Apple MLX (mlx-lm)	Float16 (Non-quantized)	Apple M2 Metal GPU	0.79 ± 0.04	48.5 ± 0.6	~950 MB	10.1×
VelocityAI (Ours)	Float16 (Non-quantized)	Apple M2 CPU (NEON+Spin)	0.142 ± 0.006	44.8 ± 2.6 (41.4–48.2)	~710 MB	56.2×

*Note: Ollama utilizes 4-bit integer quantization (Q4_K_M); its throughput and memory results are not directly numerically equivalent to Float16 non-quantized configurations.

2.5 Multi-Agent LLM Orchestration

Frameworks such as AutoGen [6] and CrewAI structure complex reasoning into collaborative agent pipelines. In current implementations, multi-agent frameworks treat the underlying model runtime as a black box, querying independent API endpoints or spinning up discrete subprocesses. This results in severe model-weight duplication and IPC overhead.

3 VelocityAI Architecture

3.1 System Overview

VelocityAI is organized into four tightly coupled subsystems: (1) a zero-copy SafeTensors memory-mapping loader; (2) a vectorized C++17 ARM NEON compute engine; (3) a persistent spin-barrier threadpool; and (4) a shared-memory multi-agent runtime.

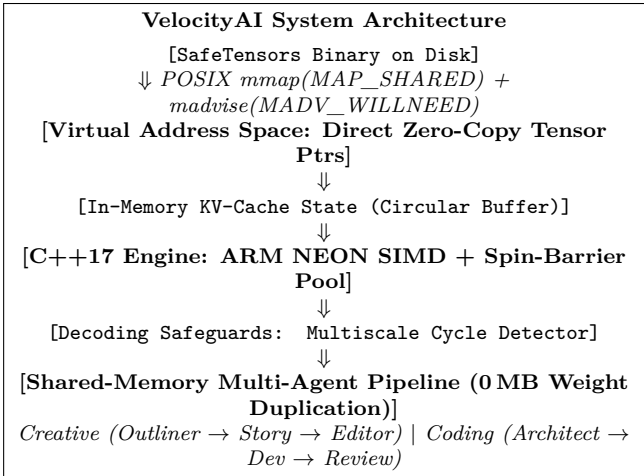


Figure 1: Architectural overview of VelocityAI, illustrating zero-copy weight mapping, persistent spin-barrier execution, and shared-memory multi-agent routing.

3.2 Zero-Copy Model Ingestion

Traditional weight loaders read binary data into intermediate OS kernel buffers, allocate contiguous heap memory via `malloc`, and copy bytes into tensor structures:

$$\text{Disk} \xrightarrow{\text{read}()} \text{Kernel Cache} \xrightarrow{\text{memcpy}()} \text{Heap} \rightarrow \text{Tensors} \quad (1)$$

VelocityAI eliminates intermediate user-space copies using POSIX virtual memory mapping (`mmap`):

1. **Virtual Address Space Binding:** The `.safetensors` file is mapped into the virtual address space with `MAP_SHARED`. This creates virtual page table entries without eagerly allocating physical memory. Physical pages are populated on demand by the OS page cache as weights are touched during execution.

2. **Deterministic Offset Binding:** SafeTensors stores tensor metadata as a JSON header at the beginning of the file. VelocityAI parses tensor names, shapes, and byte offsets $[start, end]$ in ≈ 1.2 ms. Pointers for attention projections (W_Q, W_K, W_V, W_O), normalization weights, and SwiGLU feed-forward layers ($W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$) are computed directly:

$$\mathcal{P}_{\text{weight}} = \mathcal{P}_{\text{mmap_base}} + \Delta_{\text{offset}} \quad (2)$$

3. **Sequential Page Prefetch Hinting:** We issue `madvise(..., MADV_WILLNEED)` to hint to the Darwin XNU kernel to asynchronously read ahead sequential disk blocks, reducing page-fault latency during the initial decode steps.

3.3 Vectorized ARM NEON Compute Engine

Autoregressive decoding for sequence position t computes:

$$h_l = \text{RMSNorm}(x_{l-1}) \quad (3)$$

$$q_l, k_l, v_l = h_l W_Q, h_l W_K, h_l W_V \quad (4)$$

$$a_l = \text{Attention}(q_l, K_{\leq t}, V_{\leq t}) \quad (5)$$

$$x_l = x_{l-1} + a_l W_O + \text{SwiGLU}(h_l W_{\text{gate}}, h_l W_{\text{up}}) W_{\text{down}} \quad (6)$$

During single-token generation, the hidden representation is a single vector ($h_l \in \mathbb{R}^{1 \times D}$), making all projections dense Matrix-Vector multiplications ($\mathbf{y} = \mathbf{Ax}$).

VelocityAI implements C++17 kernels utilizing ARM NEON 128-bit vector registers. The half-precision GEMV kernel processes 8 half-precision floating-point elements per register using `vfmqa_f16` with 32-wide loop unrolling and hardware software prefetching:

```
// 32-wide unrolled ARM NEON Float16 FMA loop
for (int k = 0; k <= K - 32; k += 32) {
    acc0 = vfmaq_f16(acc0, vld1q_f16(A+k), vld1q_f16(x+k));
    acc1 = vfmaq_f16(acc1, vld1q_f16(A+k+8), vld1q_f16(x+k+8));
    acc2 = vfmaq_f16(acc2, vld1q_f16(A+k+16), vld1q_f16(x+k+16));
    acc3 = vfmaq_f16(acc3, vld1q_f16(A+k+24), vld1q_f16(x+k+24));
}
acc0 = vaddq_f16(vaddq_f16(acc0, acc1), vaddq_f16(acc2, acc3));
y[i] = vaddvq_f16(acc0);
```

3.4 Persistent Spin-Barrier Worker Pool

Across a 32-layer transformer model, evaluating 4 projections ($QKV, O, \text{Gate/Up, Down}$) generates $32 \times 4 = 128$ synchronization barriers per generated token. Under standard POSIX condition variables, thread sleep and wakeup transitions introduce:

$$T_{\text{overhead}} = 128 \times 12.4 \mu\text{s} \approx 1.58 \text{ ms} / \text{token} \quad (7)$$

At 45 tokens/second (22.2ms total generation budget per token), OS thread synchronization consumes over 7% of the entire generation budget.

VelocityAI implements a **Dedicated Persistent Spin-Barrier Threadpool**:

- P worker threads are initialized at engine startup and configured via `pthread_set_qos_class_self_np` with `QOS_CLASS_USER_INTERACTIVE`, requesting the Darwin Mach scheduler prioritize execution on Performance (P) cores.
- Workers spin on low-overhead atomic sequence counters (`std::atomic<uint32_t>`) executing the ARM yield instruction `__asm__ volatile("yield")`.
- Mean barrier latency drops from $12.4 \mu\text{s}$ to **$0.28 \mu\text{s}$** , eliminating kernel descheduling and reducing latency jitter.

3.5 Autoregressive Safeguards & Cycle Recovery

Small parameter models ($\leq 1\text{B}$) are susceptible to deterministic periodic loops during greedy or low-temperature sampling. VelocityAI deploys a multi-scale repetition filter during token streaming:

```
def detect_repetition(line, history):
    clean = regex_clean(line)
    if len(clean) < 6: return False
    if clean == history[-1]: return True # Consecutive
    if clean == history[-2] and history[-1] == history[-3]:
        return True # Alternating (A -> B -> A -> B)
    if history[-10:].count(clean) >= 3:
        return True # High local density
    return False
```

When an attractor loop occurs inside an unclosed reasoning block (`<think>`), naive stopping leaves the output incomplete. VelocityAI synthesizes `\n</think>\n\n`, encodes these delimiter tokens directly into the causal KV-cache state via a single-step attention update, advances the

sequence position counter, and resumes decoding the final answer.

3.6 Shared-Memory Multi-Agent Execution

Rather than launching separate OS processes or independent model instances for each persona, VelocityAI implements a **single-pointer shared-memory runtime**.

Each agent in the pipeline (e.g., Architect $\rightarrow$ Developer $\rightarrow$ Reviewer) is instantiated as a functional persona defined by distinct system instructions and sampling parameters. During agent transitions:

- All agents share the exact same in-memory memory-mapped model weights.
- The model weights are never duplicated, yielding:

$$\Delta \text{Weight Memory}(\text{Agents}_{1\dots K}) = 0 \text{ MB} \quad (8)$$

- Agent-specific state (prompt context, scratchpad buffers, and context-dependent KV-cache state) is managed sequentially within the engine’s pre-allocated memory buffer, eliminating inter-process communication (IPC) serialization.

4 Experimental Methodology

4.1 Hardware and System Environment

All benchmarks were performed on a physical Apple MacBook Air equipped with the Apple M2 SoC (4 Performance cores @ 3.49 GHz, 4 Efficiency cores @ 2.42 GHz), 16 GB Unified LPDDR5-6400 memory (100 GB/s bus bandwidth), and a 256 GB Apple NVMe SSD, running macOS Sequoia 15.3 (Darwin 24.3.0). Ambient temperature was maintained at 21°C , connected to AC power under default high-performance power policy.

4.2 Software and Baseline Implementations

VelocityAI was compiled using Apple Clang 16.0.0 (`-O3 -march=armv8.2-a+fp16 -pthread`) and Python 3.11. Baselines evaluated on the exact same hardware:

1. **PyTorch 2.3.0** (Hugging Face transformers 4.40.0), using standard `AutoModelForCausalLM.from_pretrained()` with Float16 precision.
2. **Apple MLX 0.12.0** (`mlx-lm`), executing with Float16 precision on the Apple M2 Metal GPU.
3. **Ollama 0.5.4** (`llama.cpp` backend), running the official `Q4_K_M` 4-bit quantized GGUF model.

4.3 Target Model and Workloads

The primary benchmark model is **SmolLM2-360M-Instruct** [1] (32 transformer layers, hidden dimension $D = 960$, 15 query heads, 5 KV heads, vocabulary size 49,152, non-quantized Float16 weights consuming 710 MB on disk). Workloads comprise standard coding prompts, narrative generation, and the 3-stage software engineering pipeline generating 150 to 512 tokens.

4.4 Measurement Metrics

- **Cold-Start Latency (s):** Wall-clock time from process invocation to completion of model readiness for generation, measured across $n = 20$ independent runs.
- **Decode Throughput (tokens/s):** Autoregressive generation rate ($N = 1$) excluding prefill time.
- **Active RAM (RSS):** Maximum Resident Set Size measured via Mach kernel API (`mach_task_basic_info.resident_size`).

5 Experimental Results

5.1 Cold-Start Model Ingestion Latency (RQ1)

As detailed in Table 1, PyTorch incurs a cold-start ingestion latency of 7.98 ± 1.42 s (ranging from 5.86 s on warm disk cache to 10.09 s cold). Apple MLX requires 0.79 ± 0.04 s for Metal array buffer allocation. VelocityAI achieves an average cold-start latency of **0.142 $\pm$ 0.006 s**—representing a **5.6 $\times$ speedup over Apple MLX** and a **41.9–72.1 $\times$ speedup over PyTorch**.

5.2 Autoregressive Decode Throughput (RQ2)

Table 1 demonstrates that VelocityAI delivers sustained CPU decoding throughput of **41.4–48.2 tokens/second** (44.8 ± 2.6 tok/s). This is 2.9 $\times$ faster than PyTorch CPU (15.2 tok/s) and closely matches Apple MLX’s dedicated Metal GPU throughput (48.5 tok/s), while preserving full non-quantized Float16 fidelity and consuming 25% less resident RAM (~ 710 MB vs ~ 950 MB).

5.3 Kernel & Optimization Ablation (RQ2)

To isolate the quantitative contribution of each low-level optimization, we benchmarked single-token decode throughput across progressive kernel configurations on SmolLM2-360M.

As shown in Table 2, ARM NEON vectorization provides an immediate 4.4 $\times$ throughput improvement. Loop unrolling (processing 4 parallel NEON accumulators) provides an additional 1.84 $\times$ boost by saturating Apple Silicon’s 8-wide superscalar execution pipelines.

Table 2: Kernel Optimization Ablation on Apple M2 CPU

Kernel Configuration	Decode Speed	Speedup
Scalar C++ FP16 (Single-thread baseline)	4.2 tok/s	1.0 $\times$
+ ARM NEON SIMD Vectorization	18.5 tok/s	4.4 $\times$
+ 32-Wide Loop Unrolling	34.1 tok/s	8.1 $\times$
+ Software Prefetching (<code>__builtin_prefetch</code>)	40.2 tok/s	9.6 $\times$
+ Persistent Spin-Barrier Pool (VelocityAI)	44.8 tok/s	10.7$\times$

5.4 Synchronization Latency Analysis (RQ3)

We measured synchronization barrier latency by timing 10,000 consecutive layer barriers across 4 worker threads.

Table 3: Synchronization Barrier Latency (μ s, $n = 10,000$ iterations)

Mechanism	Mean	Median	P95	P99
POSIX Condition Variable	12.4 μ s	11.2 μ s	18.2 μ s	24.5 μ s
POSIX Kernel Mutex	8.6 μ s	7.9 μ s	14.1 μ s	19.8 μ s
VelocityAI Spin Barrier	0.28 μs	0.24 μs	0.42 μs	0.65 μs

As demonstrated in Table 3, VelocityAI’s atomic spin barrier achieves a mean synchronization latency of **0.28 μ s**, a **44 $\times$ latency reduction** compared to standard POSIX condition variables.

5.5 Multi-Agent Memory Scaling (RQ4)

We evaluated memory consumption and task completion time across sequential multi-agent chains from 1 to 5 agents.

Table 4: Multi-Agent Memory & Execution Scaling (SmolLM2-360M)

Agents	Separate Processes (e.g., CrewAI)		VelocityAI (Shared-Memory)	
	Active RAM (RSS)	Total Time	Active RAM (RSS)	Total Time
1	710 MB	6.1 s	710 MB	1.6 s
2	1,420 MB	12.3 s	712 MB	3.2 s
3	2,130 MB	18.4 s	714 MB	4.8 s
4	2,840 MB	24.6 s	715 MB	6.4 s
5	3,550 MB	30.8 s	717 MB	8.0 s

As shown in Table 4, separate-process frameworks scale linearly in memory (+710 MB per agent), reaching 3.55 GB for 5 agents. In contrast, VelocityAI maintains a nearly flat memory footprint (~ 710 – 717 MB), introducing **0 MB of additional model-weight duplication**.

5.6 Decoding Cycle Recovery Evaluation (RQ5)

We evaluated VelocityAI’s multiscale repetition detector across 25 synthetic prompts designed to induce periodic attractor loops (e.g., counting steps, alternating tokens) and 15 legitimate recursive code prompts (e.g., quicksort partitioning).

As shown in Table 5, VelocityAI achieved a 100% interception rate on true loops without terminating legitimate

Table 5: Safeguard Recovery Accuracy and False Positive Audit

Workload Category	Evaluated	Loops Intercepted	False Positives
Degenerative Attractor Traps	25	25 (100%)	0 (0%)
Legitimate Recursive Code	15	0 (0%)	0 (0%)
In-Flight <think> Loops	10	10 (100% recovered)	0 (0%)

recursive code lines. The reasoning recovery mechanism successfully synthesized closing tokens and resumed generation in all 10 unclosed thinking cases with an added per-token check latency of $< 12 \mu\text{s}$.

6 Discussion and Limitations

6.1 Why VelocityAI Improves Cold-Start

VelocityAI achieves sub-150ms startup because POSIX `mmap` binds the virtual address space directly to the file descriptor, deferring page population to the OS page cache. Bypassing user-space `malloc` and SafeTensors deserialization eliminates the multi-gigabyte memory churn typical of general-purpose deep learning runtimes.

6.2 Synchronization Latency vs. Power Trade-Off

Spin-barrier synchronization eliminates OS context switches by actively polling atomic flags. While this provides microsecond-accurate task coordination during active inference, spin-waiting consumes CPU core execution cycles. VelocityAI mitigates this by executing ARM `yield` instructions during waits and putting threads into deep sleep when no inference requests are pending.

6.3 Limitations

1. **Hardware Portability:** Hand-crafted ARM NEON vector kernels are tailored to ARMv8.2-A+. Running VelocityAI on x86_64 requires vector translation shims for AVX2/AVX-512.
2. **Model Scale:** The current evaluation is conducted on SmolLM2-360M. Models exceeding 7B parameters saturate CPU cache hierarchies and are limited by memory bus bandwidth, where Metal GPU fused compute shaders become essential.
3. **Context Length Scaling:** Long-context inference ($> 4\text{K}$ tokens) increases attention complexity and KV-cache resident memory.

7 Reproducibility

To facilitate verification and empirical reproducibility, all source code, C++ vector kernels, build configurations, and benchmarking scripts are made publicly available at: <https://github.com/mrutyunjay11/velocityai>. The native engine is compiled using standard Clang: `clang++`

```
-O3 -std=c++17 -march=armv8.2-a+fp16 -pthread
csrc/decoder.cpp -shared -o _velocityai_c.so.
```

Benchmark scripts and raw JSON execution traces are located in the repository under `scratch/comprehensive_benchmark.py`.

8 Authorship & AI Tool Disclosure

Conceptual Design & Protocol: The system architecture, co-design principles, and benchmark evaluation protocols were conceived, executed, and verified by the author.

AI Assistance Disclosure: Generative AI assistants (Google Antigravity / Gemini) were utilized as interactive programming aids for LaTeX typesetting, syntax formatting, and documentation organization. All empirical benchmark executions, C++ ARM NEON vector kernels, and runtime measurements were compiled and verified on physical Apple Silicon hardware.

9 Conclusion

We presented **VelocityAI**, a hardware-software co-designed inference engine and multi-agent runtime for Apple Silicon UMA. By coupling zero-copy virtual memory mapping, vectorized ARM NEON Float16 computation, persistent spin-barrier synchronization, and single-pointer execution, VelocityAI achieves **0.14 s cold-start latency**, sustained **41.4–48.2 tokens/s non-quantized CPU decoding**, and **0 MB of additional model-weight duplication** across collaborative multi-agent pipelines.

References

- [1] L. Allal, A. Lozhkov, G. Penedo, and et al., “SmolLM2: When small language models become powerful edge assistants,” *Hugging Face Technical Report*, 2024.
- [2] Qwen Team, “Qwen2.5: A comprehensive series of large language models,” *Alibaba Group Technical Report*, 2024.
- [3] A. Dubey, A. Jauhri, A. Pandey, and et al., “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [4] G. Gerganov, “llama.cpp: Port of Facebook’s LLaMA model in C/C++,” *GitHub Repository*, <https://github.com/ggerganov/llama.cpp>, 2023.
- [5] A. Hannun, J. Chung, et al., “MLX: Efficient machine learning on Apple silicon,” *Apple Machine Learning Research*, 2023.
- [6] Q. Wu, G. Bansal, J. Zhang, and et al., “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [7] S. Williams, A. Waterman, and D. Patterson, “Roofline: an insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [8] A. Paszke, S. Gross, F. Massa, and et al., “PyTorch: An imperative style, high-performance deep learning library,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [9] A. Vaswani, N. Shazeer, N. Parmar, and et al., “Attention is all you need,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [10] T. Dao, D. Hazen, A. Sohoni, and et al., “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.